

Classifying Supernovae

Daniel Heilman

1). Introduction:

Through surveys like the Sloan Digital All Sky Survey (SDSS), and others, there is an increasingly large number of samples of Supernova (SN) spectra. Formal methods in photometric classification were pioneered in the 2000s, which allowed supernovae to be classified from photometric data alone, though they generally involved human judgement, and the resources it would take to confirm the types for all of these spectroscopically would be immense. Thus, there is a space for machine learning algorithms which could step in and identify the types of SN from the light curves. This is exactly the idea that Kessler et. al. 2010 when they published their paper “*Supernova Photometric Classification Challenge*”. This challenge involves developing a machine learning algorithm that can predict a set of simulated supernovae based on their light curves. The supernova light curves are simulated in the g, r, i and z filters of the Dark Energy Survey (DES) for supernova types (Ia, Ib, Ic, and II), and they are simulated in proportion to the expected rate of real observations of the respective types. The simulations are based on real, spectroscopically confirmed light curves from the Carnegie Supernova Project (CSP), the Supernova Legacy Survey (SNLS), and the Sloan Digital Sky Survey-II (SDSS-II). In the dataset associated with this paper, there are 17,068 supernova light-curves. 1279 of them are part of a spectroscopically confirmed subset of the data which is provided for training. It is this subset which I will be using for designing my machine learning algorithm, both for training the algorithm and testing it to see how it performs. The spectroscopically confirmed subset is based on observations on a 4-meter class telescope with a limiting r-band magnitude of 21.5 and a limiting i-band magnitude of 23.5. There is an additional set of curves that includes the photometric redshift of the host galaxy, which I will not be using. Each object in the dataset must have at least one observation with a signal-to-noise ratio (S/N) above 5 (in any filter), and there must be at least 5 observations after the explosion. The dataset has a varying cadence, includes incomplete light curves, and has samples of inconsistent sizes.

TABLE 1
INTEGER CODES FOR SN TYPES.

SN-type	integer code
Ia	1
II (II _n , IIP, IIL)	2 (21, 22, 23)
Ibc (Ib, Ic)	3 (32, 33)
other	66
rejected	-1

1. Table 1 lists the SN-types included in the dataset, as well as the integer codes used in the files to identify them. Supernovae which do not come with a classification are marked with an integer code of -9.

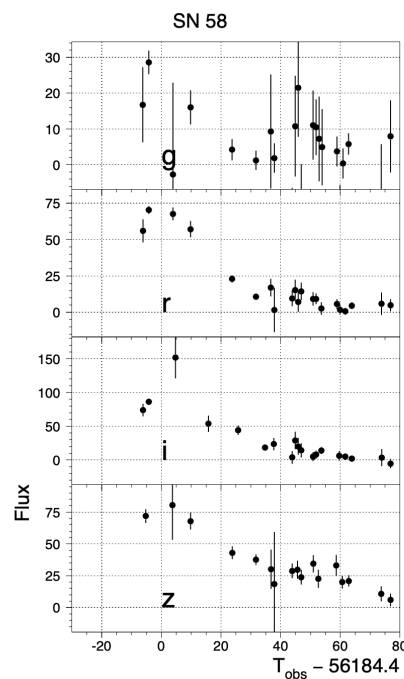
2). Approach:

In order to tackle this problem, I will be attempting to employ a Random-Forests algorithm. This algorithm is a bagging method which uses multiple decision trees, created through bootstrap sampling of

the data (the phrase bagging comes from bootstrap aggregating). The concept is to leverage different independent trees, which work in parallel. This type of model will allow for reduced variance and increased bias when compared with a single decision tree. The theory is that by utilizing a number of randomly sampled decision trees, the variance can be reduced more than the increase in bias, and thus achieving a lower test error overall. It won't have to be scaled for the data, and should be able to respond well to the variation in the lightcurves, being robust to the noise in the data. At each step, the decision trees consider different splits that could be made to the data, and choose the one that results in the lowest impurity for the result, commonly defined by the Gini impurity for classification problems. The gini impurity is what I will be using in my algorithm. Hyperparameters for this method include those for the decision trees, like the choice of impurity, as well as the number of trees, and the size of the random subset of features used at each step.

The aspects of the data I am using for my features are the time and flux pairs in each of the four bands in my dataset.

Figure 1. Example SN Lightcurve



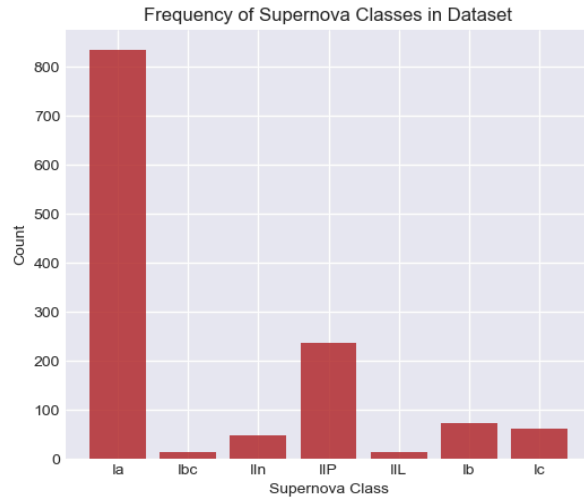
2. *Above is an example of a supernova light-curve, with time, flux pairings for each band. This supernova is not in the dataset I am using, as it is in the set containing the photometric redshifts for the host galaxy*

Since I am using scikit-learn packages for my machine learning model, I will need to transform these into a 1d array of consistent size. First, I converted all of the times into relative times. It is not important when each flux occurs in terms of its Modified Julian Date, but only when it occurs in the sequence. Thus, I have subtracted out the initial MJD value in the sequence, so that all of the time values start at one. Next, I needed to fix the issue of inconsistent lengths for the light curves between supernovae and bands. My response to this was to pad the data with false values, 999 for the time, and 0 for the flux, in order to artificially make all of the light curves the same length. Additionally, it is my hope that by using these false values instead of imputing and replacing with the mean or another measure from the

data, the model will be able to take into account the duration of the observation when assigning classes. Finally, I needed to get the four, 2d arrays into a flat 1d form, so I alternated the time, flux values with one another, and appended the resulting arrays for the filters in order g, r, i, z. Since they have all been made the same length, the features are all consistent between targets. A given index in the array corresponds to the same rank time or flux value in the same band for each target. As for the target values, these are the integer codes associated with each supernova discussed in Table 1. In the subset of the dataset I'm using, there are no unclassified (-9), type II (2), other (66), or rejected curves (-1).

Table / Figure 2. Class Distribution

SN Type	Count
Ia	834
Ibc	13
IIn	48
IIP	236
IIIL	13
Ib	73
Ic	62



- Above is the distribution of SN lightcurves in the dataset by class. By far the most prominent class is SN type Ia, followed by IIP, with the others lagging behind.*

The unequal distribution of samples will pose a challenge to my algorithm when predicting the smaller classes. To assess the results, I'll be using accuracy for the ratio correct vs incorrect, as well as a macro precision and recall statistic which will help me to determine how well I do with more than just the dominant type Ia category, which will inflate the accuracy statistic. The reason I use the 'macro' option for these settings, is that since we don't have any true negatives, as they are all just put into their classes and are in that sense true positives, I don't want to take precision and recall on a global scale. These just end up being the same as accuracy, instead we find the value by combining a version for each variable. It is notable, however, that some variables have no true positives, so their recall values will be 0, and their precision values will be undefined, and therefore arbitrarily set to 0 so that they don't contribute to the end result.

3). Initial Results:

Firstly, it is notable that, based on the distributions of the classes, we could expect a very simple classifier, which just guesses the largest, type Ia category most often, to have an accuracy of 65%. While this is very poor, and a sorry excuse for an algorithm, this is the first measure of a benchmark I can set. Initially, my first attempt at random-forests did not involve using the actual time values in my features. Instead, I just used the index for the fluxes as a proxy for the time value. Additionally, and perhaps more problematically, I left my data in repeating g, r, i, z order and padded my spectra at the end, which I don't

think allowed the data to line up properly despite being in the right order time wise. This did not work well at all, and I got quite awful results. The accuracy was 49%, even worse than the simple classifier.

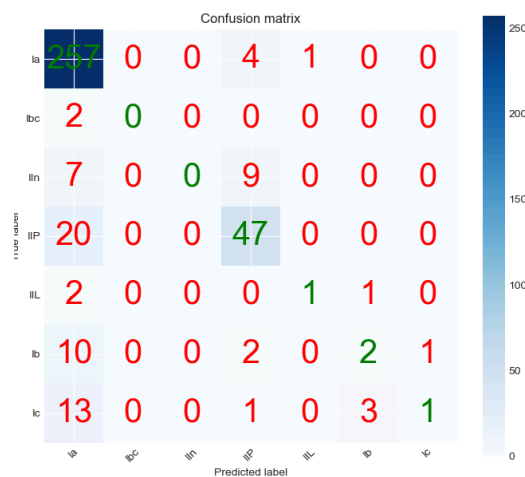
This vastly improved with my first real attempt using the complete time, flux pairs, alternating in the 1d array with each filter separated out and placed one after the other. This time, I ran a benchmark example with the default parameters for the scikit-learn RandomForest function. It performed much better, with the following results:

Benchmark Metrics Table:

Metric	Testing Value	Training Value
Accuracy	1.0	0.80
Recall ('macro')	1.0	0.30
Precision ('macro')	1.0	0.41
F statistic	1.0	0.32

Here we can already see that it is performing better than the simple classifier and certainly better than our old model. This will be what I use as the benchmark moving forwards for the new models which use the updated form of the features.

Figure 3. Benchmark Confusion Matrix



- Above is the confusion matrix for our benchmark model with all of the default parameters for RandomForest. It displays the categories assigned to each light curve by the model vs their true value, with correct predictions on the diagonal.

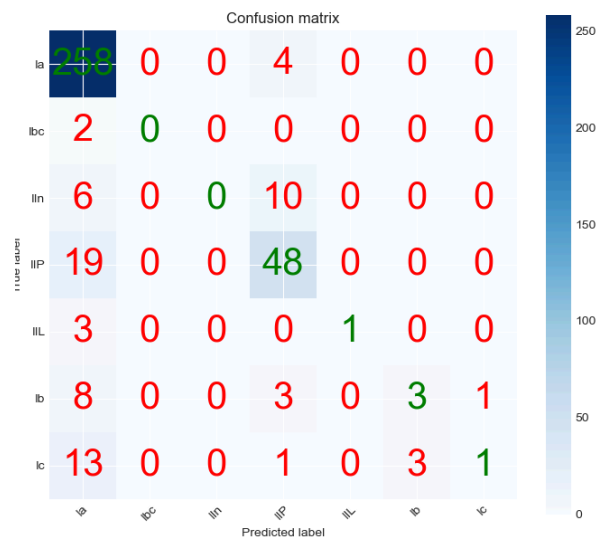
The benchmark is a good starting point, as it already has a good accuracy, but I would like to focus on increasing the precision in further iterations, so that the model is more well rounded. The accuracy statistic is inflated by the large number of correct positives in the 1a category, so increasing the precision would help us to make that more even between categories.

Beyond the initial benchmarking, I had one more initial run in which I tried playing around with the hyperparameters for the Random-Forest classifier function. I limited the max_depth in order to decrease variance, and doubled the number of estimators.

Non-Default Model Table:

Metric	Testing Value	Training Value
Accuracy	0.96	0.81
Recall ('macro')	0.88	0.32
Precision ('macro')	0.99	0.51
F statistic	0.93	0.34

Figure 4. Non-Default Model Confusion Matrix



5. Confusion matrix for the first model in which I changed hyperparameters from their defaults.

The accuracy is a tad bit lower, but we were able to increase the recall and precision values! The variance seems to have gone down, as the score for the training data is no longer 100%.

4). Final Results & Reflection:

After successfully increasing the precision of the model as desired, I attempted a couple forms of Hyperparameter optimization and cross validation to try to get an even better result. Additionally, I felt that I did not sufficiently explore the parameter space with just the one benchmark model and test run, despite playing around with a few combinations of values to get a feel for them.

First, I tried two forms of cross validation, one simply random one, and one which kept the samples representative of their distributions in the data. For the scoring for these I tried using accuracy and macro precision but neither were improved. The values for the macro precision were 0.43 and 0.40 for the first and second cross-validation techniques respectively.

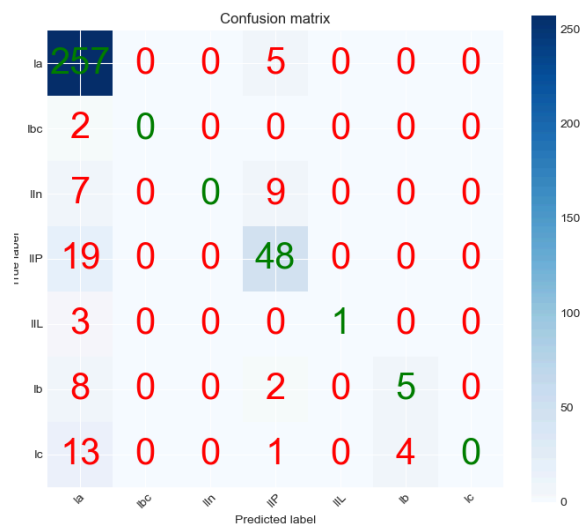
Next, instead of using this method for hyperparameter optimization, I used a grid search cross validation like the one mentioned in chapter 6. I used the following ranges of values for my parameter grid: {'max_depth': [None, 3, 5, 7, 10], 'max_features': [None, 1, 2, 5], 'min_samples_leaf': [1, 5, 10], 'n_estimators': [20, 50, 100, 200], 'criterion': ['gini']}. Using the GridSearchCV function from scikit-learn, I performed the cross validation, this time with more promising results, but still not great ones. The

results weren't too different from my non-default model, but with better Testing values and a worse training precision.

Grid-Search Model Table:

Metric	Testing Value	Training Value
Accuracy	1.0	0.81
Recall ('macro')	1.0	0.33
Precision ('macro')	1.0	0.45
F statistic	1.0	0.35

Figure 5. Grid-Search Model Confusion Matrix



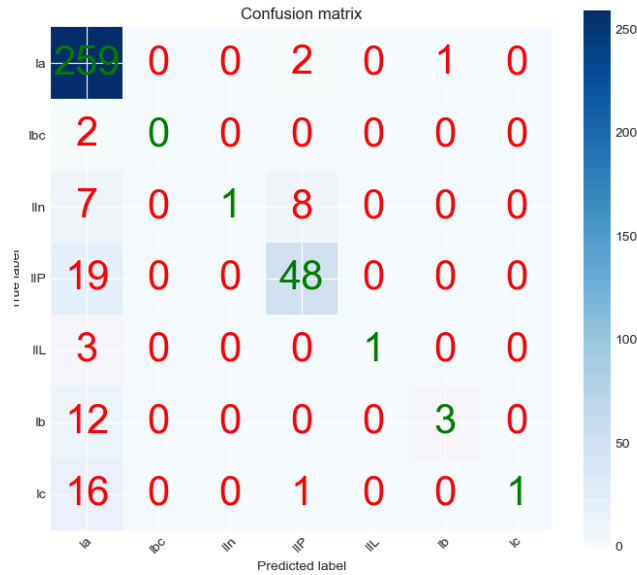
6. Confusion matrix for the grid-search model with the following parameters:
 {'criterion': 'gini', 'max_depth': None, 'max_features': None, 'min_samples_leaf': 1, 'n_estimators': 200}

Finally, I ran another model, this time changing the keyword for how the weights are handled in the RandomForestClassifier function. Since the distribution is skewed towards the type 1a, and I want to increase the precision, I opted for 'balanced' class weight this time. What this means is that the function gives a weight to the classifiers that offsets the disproportionality in the dataset. I used the same parameters as in the best grid-search model, but this time with that alteration.

Balanced Model Table:

Metric	Testing Value	Training Value
Accuracy	1.0	0.82
Recall ('macro')	1.0	0.32
Precision ('macro')	1.0	0.77
F statistic	1.0	0.37

Figure 6. Balanced Model Confusion Matrix



7. *Confusion matrix for the best grid-search model, with `class_weights='balanced'`.*

This model was the best one yet, with high testing scores, a higher accuracy than the benchmark and all the models before, and a higher precision as well. As expected, the balanced class weights greatly improved the model's precision. Additionally, as we can see in the confusion matrix, there isn't a single variable for which more of the predicted labels are incorrect than correct, hence the high precision. As for recall, there are still quite a few lightcurves being classified as type 1a, when they are in fact other types, although this model did still manage to get the most number of true positives in the type 1a category in addition to performing better in the other respects. All in all, I think this algorithm works alright for this dataset and problem but there are a couple drawbacks. I think a different algorithm could better make use of the relationship between the time and flux pairs. The recurrent neural network, for example, would be able to take into account the time dependency in the process of looping back to previous neurons, however that solution would have been more labor and computationally expensive. Regardless, I think being able to better express the (time, flux) pair would have been beneficial compared to this smoothed version. Additional steps I might take would include trying to go further with my cross-validation techniques to better explore the hyper-parameter space, implementing other methods of imputing to see if padding the data different ways has a significant effect, or if cutting the light curves down to the size of the smallest one might be a viable option as well (I'd prefer not to lose data though). Additionally, I could attempt to feature-engineer features for the dataset in addition to the light curves I have, such as a duration from continuum to peak, the value of the previous flux, or values for coefficients from a polynomial fit to the data. I think these extra features could help the model differentiate better between light-curves, and also to remove some of the dependence on where the SN falls within the window, as right now its place in the array is the most important factor of the feature. Again, these could also be addressed by using a method which is able to take into account the relationship between the feature and those around it, instead of just the feature itself. All things considered though, I think that successfully clearing the benchmark and having a relatively high accuracy and precision both above 0.75 means that my model worked reasonably well and was a success.

5). Link to Github Repo:

Linked below is the link to my github repository for the final project. In it, you'll find *Astro416_final-project*: https://github.com/heilmand/astro416_final-project

6). AI Statement:

I used chatGPT for assistance with code in two spots. The first is the function which reads in the data from a given file. I needed help with the file reader, and that function was written almost entirely by chatGPT, with me only going in to tweak a few things. I went through a couple iterations that were not written by chat, but the one I ended up using was for the most part not my work. Additionally, I needed help with a couple of lines while creating my features. In this case, chat didn't write the code for me, but helped me figure out how to pad the data with the values I wanted (np.pad was chat's idea), and helped me figure out how to use np.column_stack and np.concatenate to flatten the array.

7). References:

[1] Kessler, Conley, Jha, & Kuhlmann, (2010), [arXiv:1001.5210](https://arxiv.org/abs/1001.5210)